Revenue Agent Platform 構想整理

作成日: 2026-05-09

1. 要旨

Revenue Agent Platform は、リード獲得から提案、アウトリーチ、人間承認、決済リンク送付、着金確認までを一連のパイプラインとして自動化するAIエージェント基盤である。

当初構想では、SEO営業、人材紹介、株式自動売買など複数ドメインへの展開を想定している。現時点の実装は、その中でもビジネス化しやすく、検証しやすい「SEO営業・改善提案」の縦スライスに絞っている。

現在できていることは、低SEOスコアのWebサイトを検出し、LLMで改善提案書を生成し、SendGridでアウトリーチし、Human-in-the-loopで承認し、Stripe Payment Linkを発行して送付するところまでの主要部品である。外部APIを実際に使った本番相当のE2E運用、契約自動化、SEO改修の実行、成果報酬請求は次フェーズの範囲である。

2. 何を目指しているか

目指しているのは、単なる営業支援ツールではなく、売上につながる一連の業務をエージェントが自律的に前へ進める仕組みである。

具体的には、以下の流れを対象にしている。

フェーズ	エージェントが担うこと	現在の対象
リード探索	対象企業や機会を見つける	SEOスコアが低い企業サイトの抽出
診断・提案	課題を分析し、個別提案を作る	Lighthouse診断結果からSEO改善提案書を生成
アウトリーチ	メールやフォームで接触する	SendGridで提案メールを送信
人間承認	重要な判断を人間が確認する	Slack/URLベースのHIL承認
決済	初期費用や成果報酬を請求する	Stripe Payment Linkを生成・送付
実行	サービス提供や改善作業を行う	次フェーズ
契約	NDA、業務委託、成果報酬条件を締結する	次フェーズ

この構想の重要な前提は、完全自律ではなく「危ない判断だけ人間が止められる自律化」である。営業送信、支払いリンク送付、高額契約、投資判断などはHuman-in-the-loopを挟み、エージェントは安全に進められる範囲を自動化する。

3. 検討・検証したこと

3.1 ユースケースの検討

構想資料では、主に3つのユースケースを検討した。

ユースケース	内容	現時点の評価

SEO営業・実装エージェント	低SEOスコアのサイトを見つけ、改善提案し、初期費用を決済してもらう	MVP向き。現実の外部APIで検証しやすく、法務・規制リスクも比較的小さい
人材紹介エージェント	求人企業と候補者をマッチングし、面接調整、紹介手数料請求まで行う	価値は大きいが、個人情報、候補者同意、求人媒体規約への配慮が必要
株自動売買エージェント	市場分析、戦略立案、発注、損切り、レポートまで行う	技術的には可能だが、金融規制と損失リスクが大きく、後回しが妥当

その結果、MVPはSEO営業パイプラインに絞る判断が妥当と見ている。

3.2 アーキテクチャの検討

共通アーキテクチャとして、OpenClaw Gateway と MCP を使った外部ツール連携を想定した。エージェントの責務は、次のように分けて考えている。

役割	責務
Strategic Agent	全体の進行管理、例外処理、次アクション判断
Sales/Sourcing Agent	リード抽出、提案生成、アウトリーチ
Execution Agent	SEO改修、CMS更新、PR作成などの実作業
Finance/Legal Agent	契約、請求、決済、着金確認

現在の実装では、完全なマルチエージェント分離までは行っていない。まずは `src/pipeline/agent.ts` に各ステップを順序立ててまとめ、個別機能をモジュール化している。

3.3 技術選定の検討

領域	採用・検討した技術	理由
サイト取得	Firecrawl	JSレンダリングを含むサイト取得に対応しやすい
SEO診断	Lighthouse	SEOカテゴリのスコアと診断項目が取得できる
提案生成	Gemini + Z.ai fallback	生成コストを抑えつつ、429時に代替モデルへ逃がせる
メール送信	SendGrid	API経由で送信、ログ、リトライを扱いやすい
決済	Stripe Payment Link	Checkout構築より軽く、MVPで支払い導線を作りやすい
HIL	Slack通知 + 承認URL	人間の判断を低コストで挟める
永続化	SQLite	MVP段階で導入・運用が軽い

3.4 リスクと制御の検討

自律型の営業・決済エージェントでは、誤送信、スパム判定、不正な決済リンク送付、API障害、承認滞留が問題になる。そのため、以下を設計に入れている。

リスク	制御策
大量送信によるスパム化	1日50件の送信上限

同一企業への重複営業	30日間の重複送信防止
API失敗	リトライ、スキップ、ログ記録
承認漏れ	48時間未承認で保留化・再通知
無承認の営業・請求	HIL承認をステータス遷移に組み込む

4. どこまで出来たか

4.1 実装済みの主要コンポーネント

コンポーネント	実装内容	主なファイル
site-crawler	URLをクロールし、Lighthouse SEOスコアで低スコアサイトを抽出	`src/site-crawler/*`
proposal-generator	SEO診断結果からMarkdown提案書を生成し、PDF変換も試行	`src/proposal-generator/*`
llm-provider	Geminiをプライマリにし、429時のみZ.aiへフォールバック	`src/utils/llm-provider.ts`
outreach-sender	SendGrid送信、重複防止、日次送信上限、送信ログ	`src/outreach-sender/sender.ts`
hil-approval-flow	承認・却下URL、トークン検証、自動返信判定、タイムアウト監視	`src/hil-approval-flow/*`
stripe-payment-link	Payment Link生成、送付、リマインド、Webhook処理	`src/stripe-payment-link/*`
pipeline	クロール、提案、アウトリーチ、HIL、決済のステップ統合	`src/pipeline/agent.ts`
server	HIL承認URL、Stripe Webhook、ヘルスチェック	`src/server.ts`
storage	SQLiteによるターゲット、送信ログ、ステータス管理	`src/utils/db.ts`, `src/pipeline/state.ts`

4.2 検証済みの振る舞い

テストでは、以下の動作を確認している。

項目	確認内容	テスト
クロール結果の抽出	閾値以下のSEOスコアだけターゲット化、失敗URLのスキップ、50件上限	`tests/site-crawler.test.ts`
提案書生成	必須セクションを含む提案書生成	`tests/proposal-generator.test.ts`
LLM fallback	Gemini成功、429時のZ.ai fallback、429以外はthrow	`tests/llm-provider.test.ts`

アウトリーチ	連絡先なしのスキップ、30日重複防止、送信ログ記録	`tests/outreach-sender.test.ts`
HIL	トークン検証、承認・却下、自動返信判定	`tests/hil-approval-flow.test.ts`
決済	Payment Link生成、Webhookイベント処理	`tests/stripe-payment-link.test.ts`

4.3 現在の到達点

現時点の到達点は、「MVPの骨格実装は完了し、外部サービスをモックした単体・統合テストで主要ロジックを検証済み」という段階である。

実運用可能なプロダクトというより、デモ可能なプロトタイプに近い。外部APIキー、送信ドメイン、Stripeテスト環境、Slack通知先、Firecrawl/Lighthouse実行環境を設定すれば、1件単位のサンドボックスE2E検証に進める状態である。

4.4 まだ出来ていないこと

未完領域	内容
本番相当E2E	Firecrawl、Lighthouse、SendGrid、Slack、Stripeをすべて実APIで通す検証
返信監視の実運用	メールボックス連携、クリック計測、ポジティブ返信分類
契約自動化	DocuSign/Dropbox Sign連携、契約テンプレート、署名状況管理
SEO改修の実行	GitHub PR作成、CMS連携、実装後の再計測
成果報酬請求	成果指標の定義、計算ロジック、追加請求
管理UI	ターゲット一覧、ステータス確認、手動介入、ログ閲覧
コンプライアンス	送信同意、特商法、個人情報、媒体規約、オプトアウト処理
運用監視	ジョブ監視、失敗通知、再実行、API利用量管理

5. これからどこまで出来そうか

5.1 短期: 1〜2週間

1〜2週間で狙えるのは、1件から数件の実ターゲットに対するサンドボックスE2Eデモである。

到達イメージ:

- Firecrawl/Lighthouseで実サイトを診断する
- 低スコアサイトをターゲット化する
- LLMで提案書を生成する
- 人間が内容を確認して送信を承認する
- SendGridの認証済み送信元から提案メールを送る
- 返信または手動承認を起点にStripeテストPayment Linkを送る
- Webhookで支払い完了を検出し、ステータスを更新する

必要な作業:

- `.env` の実APIキー設定
- SendGridの送信ドメイン認証
- Stripeテスト環境とWebhook設定
- Slack通知先の設定
- 1件E2E用の運用手順整理
- 実APIでの失敗パターン修正

5.2 中期: 3〜4週間

3〜4週間で狙えるのは、限定運用できるMVPである。

到達イメージ:

- 1日10〜50件程度のターゲット抽出とアウトリーチ
- 管理者が承認・却下・保留を確認できる最低限の運用導線
- 提案書テンプレートの品質改善
- 送信ログ、重複防止、オプトアウトの運用
- Stripeテストではなく実決済の小規模検証

この段階では、まだ「完全自動営業」ではなく、「人間が最終確認する自動営業オペレーション」として扱うのが現実的である。

5.3 1〜2か月

1〜2か月で狙えるのは、SEO営業向けの実用MVPである。

到達イメージ:

- 小規模な本番営業運用
- 顧客返信の分類とHIL起動
- 決済完了後のオンボーディング通知
- 提案書PDFの安定生成
- エラー監視、再実行、失敗時の人間通知
- 基本的な管理UIまたはCLI

この段階で、実際の獲得単価、返信率、支払い率、提案書作成コスト削減効果を測れる。

5.4 3か月以降

3か月以降は、収益化の深さを上げるフェーズになる。

候補:

- GitHub PRまたはCMS連携によるSEO改修実行
- DocuSign/Dropbox Signによる契約自動化
- 改修後の再計測レポート
- 月額契約・成果報酬・追加請求
- 複数業種テンプレート
- 人材紹介など別ドメインへの横展開

株自動売買のような金融領域は、技術検証よりも先に法務・リスク・運用体制の整理が必要であるため、SEO営業MVPでエージェント基盤の収益化を検証した後に検討するのが妥当である。

6. 現時点の評価

現時点では、構想の全体像に対して「売上までの最短ルート」をSEO営業に絞り、必要なモジュールを一通り実装できている。

特に重要なのは、以下の点である。

- リード抽出、提案生成、アウトリーチ、承認、決済という収益導線が1本につながっている
- HILを前提にしているため、営業・決済のリスクを人間が制御できる
- LLM、メール、決済、Slack、SQLiteが疎結合なモジュールとして分かれている
- OpenSpec上の要件とテストが対応しており、実装範囲を説明しやすい

一方で、現段階で過大に言うべきではない点もある。

- 外部APIを実際につないだ本番E2Eは未確認
- 契約締結と実サービス提供は未実装
- 「返信を読んで商談化する」部分はまだ限定的
- 送信品質、法務、オプトアウト、ドメインレピュテーションは運用設計が必要

したがって、説明時の表現としては「構想段階」ではなく、「SEO営業に絞ったMVP実装済み。次は実APIでのE2E検証と限定運用」という言い方が最も正確である。

7. 次にやるべきこと

優先順位は以下がよい。

優先度	作業	目的
1	実APIキーを設定して1件E2Eを通す	現実の外部サービスで詰まる箇所を洗い出す
2	提案書PDF生成を安定化する	営業資料として使える品質にする
3	SendGridの送信ドメイン認証とオプトアウト導線を整える	営業運用リスクを下げる
4	管理者用の簡易ステータス確認画面を作る	HILと運用監視をしやすくする
5	返信監視とポジティブ返信分類を実装する	手動確認から商談化自動化へ進める
6	決済後オンボーディングと実作業フローを追加する	着金後の価値提供までつなげる

8. 参考にした主な資料・実装

- 構想資料: `docs/concepts/revenue-agent-platform.md`
- SEO営業パイプライン提案: `openspec/changes/archive/2026-04-24-seo-outreach-pipeline/proposal.md`
- SEO営業パイプライン設計: `openspec/changes/archive/2026-04-24-seo-outreach-pipeline/design.md`
- 実装タスク: `openspec/changes/archive/2026-04-24-seo-outreach-pipeline/tasks.md`
- LLM fallback変更: `openspec/changes/archive/2026-05-03-llm-provider-fallback/tasks.md`
- 現行仕様: `openspec/specs/*/spec.md`
- 実装コード: `src/`
- テスト: `tests/`